

R Programming Language

Outputs

- ★ text output કરીને double quotes માં text યોગ્ય જોડાઈ જાય છે.

E.x. :- "Hello World" $\Rightarrow$ [1] "Hello World"

- ★ numbers output કરીને quotes ના બદલે numbers જોડાઈ જાય છે.

E.x. :-
5 [1] 5
10 $\Rightarrow$ [1] 10
25 [1] 25
5+5 [1] 10

- ★ R ના print() function યોગ્ય જાય છે. એક યાદીમાં જોઈને આ ૨, ૨૦૦ સુધી જાય છે.

E.x. :-
print("Hello world") [1] "Hello World"
print(10) $\Rightarrow$ [1] 10

Comments

- ★ R માં એક single line અને multiline પ્રકારના comments જોડાઈને '#' જોડાઈ.

E.x. :- # This is comment.

Variables

- ★ R માં variables declare કરીને કોઈક command યોગ્ય જાય છે.
- ★ variable યોગ્ય રીતે જોડાઈને એક પ્રકારના variable name યોગ્ય જોડાઈ, એ value યોગ્ય assign કરીને પ્રકારના.

E.x. :-
name <- "Praveen"
age <- 21

name # Just output the value of variable "name"
age # Output the value of variable "age"

- ★ variable યોગ્ય value યોગ્ય assign કરીને કોઈક યોગ્ય યોગ્ય, '<-' operator યોગ્ય. એ પ્રકારના '=' operator યોગ્ય ૨ જોડાઈ જાય છે.

★ '=' 0 වඩා '<-' ආවේණික වන සමීකරණයක්. එමඟින් අගයයන් '=>' ආවේණිකයන් ලෙසින් ප්‍රකාශ කළ හැක.

★ variable value එකක් output කිරීමට variable name එකක් භාවිත කළ හැක. එමඟින් print() function එක උපයෝගී කර ගත හැක.

Ex. :- age <- 21
print(age) => [1] 21

★ strings සහ variables දෙකම variables එකතු කිරීමට concatenate කිරීමට paste() function එක භාවිත කළ හැක. එමඟින් combine කළ යුතු elements දෙකම ',' (comma) භාවිත කළ හැක.

Ex. :- 1) text <- "awesome"
paste("R is ", text)

2) text1 <- "R is "
text2 <- "awesome"
paste(text1, text2)

★ Concatenation කළහොත් '+' operator එක භාවිත කළ හැක.

★ variables එකතු කිරීමට value එක භාවිත කළ හැක.

Ex. :- var1 <- var2 <- var3 <- "Orange"

#print variable values

var1

var2

var3

Variable naming rules

- ⊛ variable name શેક્ષન ડોટથી ~~ન~~ હોઈએ english letter શેક્ષનનું હોઈ period(.) શેક્ષનનું હોઈ.
- ⊛ period(.) શેક્ષન ડોટથી શેક્ષનનું નથી; ~~શેક્ષન~~ ^{character} શેક્ષન દેખ digit શેક્ષનનું હોઈ શેક્ષન.
- ⊛ variable name શેક્ષન ડોટથી ~~ન~~ હોઈએ english letters, digits, period(.) અને underscore(-) હોઈ.
- ⊛ variable name શેક્ષન number શેક્ષનનું હોઈ underscore શેક્ષન ડોટથી શેક્ષન હોઈ.
- ⊛ variable names, case sensitive હોઈ.
- ⊛ variable names હોઈ reserved words હોઈ શેક્ષન.

Data types

- ⊛ R હોઈ variables શેક્ષનનું હોઈ; data type શેક્ષન શેક્ષનનું હોઈ હોઈ assign શેક્ષન value શેક્ષન ડોટથી data type શેક્ષન શેક્ષન હોઈ.
- ⊛ શેક્ષન value શેક્ષન assign શેક્ષનનું ડોટથી હોઈ, data type શેક્ષન શેક્ષન હોઈ.

E.X. :- my_var <- 30 # type is 'numeric'

my_var <- "Hello" # type is 'character' (a.k.a. string)

- ⊛ R હોઈ data types શેક્ષન.

* numeric → 10.5, 55, 787

* integer → 1L, 55L ⇒ letter "L" declares this as an integer

* complex → 9+3i ⇒ "i" is the imaginary part.

* character (a.k.a. string) → "R is exciting"

* logical (a.k.a. boolean) → TRUE, FALSE

- ⊛ variable શેક્ષન data type શેક્ષન check શેક્ષન class() function શેક્ષન હોઈ.

E.X. :- x <- 1000L

class(x)

⇒ "integer"

★ R 2 number types 3 කි.

1) numeric $\Rightarrow$ දශම කොටස හෝ දුර්ලභ කොටස store කළ හැක.

2) integer $\Rightarrow$ දුර්ලභ කොටස පමණක් store කළ හැක. integer value එකින් ආසන්නයට "L" යෙදිය යුතුය.

3) complex $\Rightarrow$ imaginary part (denoted by 'i') එකින් කළ හැක.

★ එක් number type එකකින් තවත් එකකට value එකතු type එක convert කිරීම සඳහා as.numeric() යොදා ගත හැක.

* as.numeric()

* as.integer()

* as.complex()

Ex. :- `x <- 1L #integer`
`y <- 2 #numeric`

`a <- as.numeric(x)`

`b <- as.numeric(y)`

`class(a)`

`class(b)`

Math functions

★ min() මගින් number set එකතු minimum value එක සහ max() මගින් number set එකතු maximum value එක ලබාගත හැක.

Ex. :- `max(5, 10, 15)` $\Rightarrow$ [1] 15
`min(5, 10, 15)` [1] 5

★ sqrt() මගින් වර්ගමූල ග්‍රහණය output කළ හැක.

Ex. :- `sqrt(16)` $\Rightarrow$ [1] 4

★ abs() මගින් අග්‍රහණය කළ ග්‍රහණය / මානසය ලබාගත හැක.

Ex. :- `abs(-4.7)` $\Rightarrow$ [1] 4.7

★ ceiling() මගින් ආසන්නයට ඉහළ දුර්ලභ කොටස සහ floor() මගින් ආසන්නයට පහළ දුර්ලභ කොටස වටහන් කළ හැක.

Ex. :- `ceiling(1.4)` [1] 2
`floor(1.4)` $\Rightarrow$ [1] 1

Strings

- ★ strings હોય છે single quotes અને double quotes બંનેમાં લખવામાં આવે છે.
- ★ string એકાંત multiline string એકાંત હોય છે, અર્થાત્, output એકાંત એકાંત હોય છે. lines break એકાંત એકાંત એકાંત હોય છે.
- ★ જો કોઈ line break કરવા હોય તો \n લખવામાં આવે છે, output એકાંત એકાંત cat() function હોય છે.

E.x.:- str <- "My name
is praveen
Theekshana"

print(str)
cat(str)

⇒ "My name is praveen Theekshana"

"My name
is praveen
Theekshana"

- ★ nchar() એકાંત string એકાંત જોઈ characters લેવા (length of a string) હોય છે.

E.x.:- str <- "Hello"
nchar(str)

⇒ [1] 5

- ★ character એકાંત એકાંત character sequence એકાંત એકાંત string એકાંત જોઈ એકાંત check કરવા grepl() function હોય છે.

E.x.:- str <- "Hello"

grepl("H", str)

[1] TRUE

grepl("Hello", str)

⇒ [1] TRUE

grepl("X", str)

[1] FALSE

Escape Characters

Escape character	result
\\	backdash
\n	new line
\t	tab space
\b	backspace

Operators

① କ୍ଷମା ସହ 5 ଅଟେ.

- * Arithmetic operators
- * Assignment operators
- * Comparison operators
- * Logical operators
- * Miscellaneous operators

Arithmetic Operators

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
^	Exponent	$x ^ y$
%%	Modulus	$x \% y$
%/%	Integer Division	$x \% / \% y$

Ex:- ① $2 ^ 5$ # Same as $2 * 2 * 2 * 2 * 2$

↓
[1] 32

② $5 \% 2$

↓
[1] 1

③ $15 \% / \% 2$

↓
[1] 7

integer division rounds the result down to the nearest whole number.

Assignment operators

$\left. \begin{array}{l} <- \\ \rightarrow \end{array} \right\} \text{local assigner}$

$\left. \begin{array}{l} \ll- \\ -\gg \end{array} \right\} \text{global assigner}$

E.x.:- $\text{my_var} <- 3$

$\text{my_var} \ll- 3$

$3 \rightarrow \text{my_var}$

$3 -\gg \text{my_var}$

Ⓐ $x <- 3$ is equal to $3 \rightarrow x$

Comparison operators

operator	name
$==$	Equal
$!=$	Not equal
$>$	greater than
$<$	less than
$>=$	greater than or equal to
$<=$	less than or equal to

Logical operators

operator	Description
&	Element-wise Logical AND operator. Returns TRUE if both elements are true.
&&	Logical AND operator. Returns TRUE if both statements are true.
	Element-wise Logical NOT operator. Returns TRUE if one of the statements is TRUE.
	Logical OR operator. Returns TRUE if one of the statements is TRUE.
!	Logical NOT operator. Returns FALSE if statement is TRUE.

Miscellaneous operators

* Use to manipulate data.

operator	Description	Example
:	creates a series of numbers in a sequence	$x \leftarrow 1:10$
% in %	Find out if an element belongs to a vector	$x \% \text{in} \% y$
% * %	Matrix multiplication	$x \leftarrow \text{Matrix1} \% * \% \text{Matrix2}$

If - else - else if

Example ;

```
a <- 200
```

```
b <- 30
```

```
if (b > a) {  
  print("b is greater than a")  
}  
else if (a == b) {  
  print("a and b are equal")  
}  
else {  
  print("a is greater than b")  
}
```

Loops

while loops

Example ;

```
i <- 1  
while (i < 6) {  
  print(i)  
  i <- i + 1  
}
```

output
⇒

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

break

Example ;

```
i <- 1  
while (i < 6) {  
  print(i)  
  i <- i + 1  
  if (i == 4) {  
    break  
  }  
}
```

Output
⇒

```
[1] 1  
[1] 2  
[1] 3
```

Next

Example;

```
i <- 0
while (i < 4) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

Output
⇒

[1] 1

[1] 2

For loop

⊛ use to iterating over a sequence.

Example;

```
for (x in 1:10) {
  print(x)
}
```

⊛ we can execute a set of statements, once for each item in vector, array, list ... etc.

Example 1 ⇒ print every item in a list.

```
fruits <- list("apple", "banana", "cherry")
for (x in fruits) {
  print(x)
}
```

Example 2 ⇒ print the number of dices.

```
dice <- c(1, 2, 3, 4, 5, 6)
for (x in dice) {
  print(x)
}
```

Functions

- ① To create a function, use "function()" keyword.

Example;

```
my_function <- function() {  
  print("Hello World")  
}
```

- ② To call a function, use the function name followed by parentheses.

Example; my_function() ^{output} ⇒ [1] "Hello World"

- ③ When we declare a function, we can use parameters inside the parentheses.

Then, when we call that function we should pass data into that function as arguments.

Example; my_function <- function(fname) {
 paste(fname, "Griffin")
}

[1] "Peter Griffin"

my_function("Peter")

⇒
Output

[1] "Lois Griffin"

my_function("Lois")

- ④ We can use default parameter value as we want.

Example; my_function <- function(country = "Norway") {
 paste("I am from", country)
}

my_function("Sweden")

[1] "I am from Sweden"

my_function()

⇒
Output

[1] "I am from Norway"

⊛ If we want to return a value from a function, we use "return()" function.

```
Example; my-function <- function(x) {  
  return(5*x)  
}  
print(my-function(3))  
print(my-function(5))
```

[1] 15
[2] 25
⇒
Output

Nested functions

⊛ There are two ways to create nested function.

i.) call a function within another function

ii.) write a function within a function

```
i.) Nested-function <- function(x,y) {  
  a <- x+y  
  return(a)  
}
```

[1] 10
⇒
Output

```
nested-function(Nested-function(2,2), Nested-function(3,3))
```

ii.) ~~var~~ Outer-function <- function(x) {

```
  Inner-function <- function(y) {
```

```
    a <- x+y
```

```
    return(a)
```

```
  }
```

```
  return(Inner-function)
```

```
}
```

```
output <- Outer-function(3)
```

```
output(5)
```

[1] 8
⇒
Output

Recursion

* In R language, we can use the concept called 'function recursion'.

* Recursion means, defined function can call itself.

Example: `tri_recursion <- function(k) {`

`if (k > 0) {`

`result <- k + tri_recursion(k-1)`

`print(result)`

[1] 1

`}`

[1] 3

`else {`

`result = 0`

`return(result)`

[1] 6

[1] 10

[1] 15

`}`

`}`

[1] 21

`tri_recursion(6)`

Global variables

① function એક્સર્સિઝ ફોલો હોઈ શકે છે, variable એક્સર્સિઝ "global variable" એક્સર્સિઝ હોઈ શકે છે.

② Global variables, function એક્સર્સિઝ સુધી હોઈ શકે છે અને ફોલો હોઈ શકે છે.

Example: `txt <- "awesome"`

`my_function <- function() {`

`paste("R is", txt)`

`}`

`my_function()`

③ R સુધી હોઈ શકે છે global અને local variables એક્સર્સિઝ ફોલો હોઈ શકે છે.

Example; `txt <- "global variable"`

```
my-function <- function() {  
  txt = "fantastic"  
  paste("R is ", txt)
```

[1] "R is fantastic"

```
}
```

⇒ [1] "global variable"

```
my-function()
```

```
txt
```

Output

- ⊛ When we use "global assignment operator" (`<-`), to assign a value to a local variable; it belongs to global scope.

Example; `my-function <- function() {`
`txt <- "fantastic"`
`paste("R is ", txt)`

[1] "R is ^{fantastic}~~awesome~~"

```
}
```

⇒ [1] "fantastic"

```
my-function()
```

```
print(txt)
```

Output

- ⊛ If we want to reassign the value of a global variable, inside a function; we have to use global assignment operator.

Example; `txt <- "awesome"`

```
my-function <- function() {  
  txt <- "fantastic"  
  paste("R is ", txt)
```

[1] "R is fantastic"

```
}
```

[1] "R is fantastic"

```
my-function()
```

```
paste("R is ", txt)
```

⇒
Output

R Data Structures

Vectors

* වෛක් data type වෛක් items list වෛක්.

* vector වෛක් අර්ථයක් c() function වෛක් වෛක් items, comma seperated list වෛක් වෛක් වෛක්.

E.x. :- #vector of strings

```
fruits <- c("banana", "apple", "orange")
```

```
fruits # print fruits
```

↓ output

```
[1] "banana" "apple" "orange"
```

* numerical value sequence වෛක් vector වෛක් assign කිරීමට ":" operator වෛක්.

E.x. numbers <- 1:10

```
numbers
```

⇒
output

```
[1] 1 2 3 4 5 6 7 8 9 10
```

* decimals අන්තර්ගත numerical values sequence වෛක් vector වෛක් assign කළ හැක. වෛක් වෛක් අන්තර්ගත element වෛක් sequence වෛක් අන්තර්ගත වෛක් වෛක් වෛක්.

E.x. :- numbers1 <- 1.5 : 6.5

```
numbers1
```

output

⇒

```
[1] 1.5 2.5 3.5 4.5 5.5 6.5
```

```
numbers2 <- 1.5 : 6.3
```

```
numbers2
```

output

⇒

```
[1] 1.5 2.5 3.5 4.5 5.5
```

* TRUE, FALSE අන්තර්ගත sequence වෛක් වෛක්, vector වෛක් assign කළ හැක.

```
log_values <- c(TRUE, FALSE, TRUE)
```

* Vector length (number of elements) - ଏହା `length()` function ଦ୍ୱାରା କରାଯାଇଥାଏ।

E.X. `fruits <- c("banana", "apple", "orange")`

`length(fruits)`

⇒
output [1] 3

* alphabetically and numerically, vector items sort କରିବା `sort()` function କରାଯାଇଥାଏ।

E.X. `fruits <- c("banana", "apple", "mango")`

`numbers <- c(13, 3, 5, 7)`

`sort(fruits)`

`sort(numbers)`

⇒
output

[1] "apple" "banana"
"mango"

[1] 3 5 7 13

* Vector elements access କରିବା indexing କରାଯାଇଥାଏ। Square brackets ବ୍ୟବହାର କରି।

E.X. `fruits <- c("banana", "apple")`

`fruits[1]` # Access the first item (banana)

① Note: R ภาษामें indexing शुरुआत 1 से।

first item ⇒ index 1

* दोनो index positions पर दुनो items 'access' करके `c()` function द्वारा square brackets में लिखें।

E.X. `fruits <- c("banana", "apple", "orange")`

`fruits[c(1,3)]` # access the first and third element
banana and orange

* negative indexing से element दोनो side से access कर सकते हैं।

E.X. `fruits[c(-1)]` # Access all items except the first item

* item दोनो side में change करने के लिए indexing का उपयोग करें।

E.X. `fruits[1] <- "pear"`

change "banana" to "pear"

* vector එකක් repeat කිරීමට rep() function යොදා. (repeat each value)

E.x. repeat_each <- rep(c(1,2,3), each=2)

repeat_each

↓ output

[1] 1 1 2 2 3 3

* sequence එක repeat කිරීමට &, මෙය භාවිත කළ හැක.

E.x. repeat_times <- rep(c(1,2,3), times=2)

repeat_times

↓ output

[1] 1 2 3 1 2 3

* එක් එක් value එක independently repeat කළ හැක.

E.x. repeat_independently <- rep(c(1,2,3), times=c(5,2,1))

repeat_independent

* වඩා විශාල ලෙස ක්‍රියා steps නිර්මාණය කළයුතු විට sequence එකක් නිර්මාණය කිරීමට seq() function යොදා.

E.x. numbers <- seq(from=0, to=100, by=20)

numbers

↓ output

[1] 0 20 40 60 80 100

Lists

* list එකක විවිධ data types වල data ගබඩා කළ හැක.

* Ordered නො changeable වේ.

* list එකක් නිර්මාණයට list() function යොදා.

E.x. thislist <- list("apple", "banana", "orange")

thislist

⇒

output

[[1]]

[1] "apple"

[[3]]

[1] "orange"

[[2]]

[1] "banana"

* list items access කිරීම indexing කෙරෙයි.

E.x. thislist [1] $\Rightarrow$ [1]
output [1] "apple"

* list item value වෙනස් change කිරීම ද, indexing කෙරෙයි.

E.x. thislist [1] <- "peach"

* list length වෙන (number of list items) මොහොතකදී length() function කෙරෙයි.

E.x. length(thislist) $\Rightarrow$ output [1] 3

* කිසිකිසි item වෙනස් list වෙනස් කිරීමට "%in%" operator කෙරෙයි.

E.x. "apple" %in% thislist $\Rightarrow$ output [1] TRUE

* list වෙනස් කිරීම item වෙනස් add කිරීම append() function කෙරෙයි.

E.x. append(thislist, "cherry")

* විශේෂ index වෙනස් item වෙනස් place කිරීම ද, append() function කෙරෙයි නැත. මෙය ද after keyword වෙනස් කරන index වෙනස් කිරීම ලෙස.

E.x. append(thislist, "peach", after=2)

#add "peach" to the list after "banana"

* list items remove කිරීම indexing කෙරෙයි.

E.x. thislist <- list(1, 2, 3, 4) [1]
newlist <- thislist[-1] [1] 2
newlist $\Rightarrow$ [2]
output [1] 3
[3]
[1] 4

* indexes range වෙනස් කිරීම output කිරීම items වෙනස් කිරීම ද, indexes range කිරීම.

E.x. thislist <- list(1, 2, 3, 4) [1]
(thislist) [2:4] $\Rightarrow$ output [1] 2
[2] 3
[3] 4
[1] 4

* list એક્સેસ થી બે જાણે combine કરીને c() function લેવામાં આવે છે.

E.x. list1 <- list(1,2,3)

list2 <- list(10,20,30)

list3 <- c(list1, list2)

list3

Matrices

* Matrix એક 2D data set છે. columns અને rows યોગ્ય રીતે ગણવામાં આવે છે. matrix() function નો use કરીને rows અને columns નો સ્પષ્ટ કરવામાં આવે છે.

E.x. thismatrix <- matrix(c(1,2,3,4,5,6), nrow = 3, ncol = 2)

thismatrix

↓ output

	[,1]	[,2]
[1,]	1	4
[2,]	2	5
[3,]	3	6

* items access કરીને square brackets નો use કરવામાં આવે છે. 1st row અને column numbers નો સ્પષ્ટ કરવામાં આવે છે.

E.x. thismatrix <- matrix(c(1,2,3,4), nrow = 2, ncol = 2)

thismatrix [1,2]

↓ output

[1] 3

* જો 1st row અને column નો સ્પષ્ટ કરવામાં આવે છે તો output આ મુજબ આવે છે.

E.x. thismatrix [1,] # whole 1st row

thismatrix [,2] # whole 2nd column

* જો 1st row અને column નો સ્પષ્ટ કરવામાં આવે છે તો output કરીને c() function નો use કરવામાં આવે છે.

E.x. thismatrix <- thismatrix(c(1,2,3,4,5,6), nrow = 3, ncol = 3)

thismatrix [c(1,2),]

thismatrix [, c(1,2)]

* matrix එකතු කිරීම columns එක් කිරීමට cbind() function ඔබට දීමට rows එක් කිරීමට rbind() function ඔබට.

E.x. newmatrix1 <- ~~thismatrix~~ cbind(thismatrix, c(1,2,3))
newmatrix2 <- rbind(thismatrix, c(1,2,3))

* matrix එකතු rows ඔබට columns remove කිරීමට c() function ඔබට.

E.x. thismatrix <- thismatrix [-c(1), -c(1)]
thismatrix

* කිසිදු item එකතු කිරීමට ඔබට “%in%” operator ඔබට.

E.x. 1 %in% thismatrix ⇒ output [1] TRUE

* dim() function ඔබට number of rows and columns ඔබට දීමට.

E.x. dim(thismatrix) ⇒ output [1] 2 2

* length() function ඔබට matrix එකතු elements ගණන ඔබට දීමට.

E.x. length(thismatrix) ⇒ output [1] 4

* matrix combine කිරීමට rbind() ඔබට cbind() functions ඔබට.

E.x. matrix-combined <- rbind(matrix1, matrix2)
matrix-combined <- cbind(matrix1, matrix2)

Arrays

* Arrays වල dimensions දීමට ඔබට ඔබට දීමට.

* array එකතු කිරීමට array() function ඔබට. ‘dim’ parameter එක ඔබට dimensions specify කිරීමට.

E.x. thisarray <- c(1:24)

thisarray

multiarray <- array(thisarray, dim = c(4,3,2))

multiarray

Ⓢ Note : dim = c(4,3,2) හි 4,3,2 යනු rows ඔබට columns ගණන දීමට. dimensions ගණන දීමට.

Array එකතු කිරීමට ඔබට data type එකතු කිරීමට.

* Array elements access કરીને indexing કરવામાં આવે છે.

E.X. multiarray [2,3,2]

① Syntax → array_name [row position, column position, matrix level]

* જોઈએ તો row એક્સેસ કરીને column એક્સેસ કરવામાં આવે છે access કરીને ૨, શ્રેણીમાં મૂકે.

E.X. multiarray [c(1),,1]

multiarray [,c(1),1]

* Element એક્સેસ કરવામાં જોઈએ "%in%" operator વાપરવામાં આવે છે.

E.X. 2 %in% multiarray

* rows અને columns બંનેના output કરીને dim() function વાપરવામાં આવે છે.

E.X. dim(multiarray)

* array dimension એક્સેસ કરીને length() function વાપરવામાં આવે છે.

E.X. length(multiarray)

Data Frames

* table એક્સેસ કરવામાં data display કરીને કરવામાં આવે છે.

* Data frame એક્સેસ કરીને data types પર data પ્રાપ્ત કરવામાં આવે છે.
અર્થમાં; એક્સેસ કરવામાં આવે છે એક્સેસ કરવામાં આવે છે type એક્સેસ કરવામાં આવે છે.

* data.frame() function વાપરવામાં data frame બનાવવામાં આવે છે.

E.X. Data-frame <- data.frame(

Training = c("Strength", "Stamina", "Other"),

Pulse = c(100, 150, 120),

Duration = c(60, 30, 15)

)

Data-frame

⇓ output

	Training	Pulse	Duration
1	Strength	100	60
2	Stamina	150	30
3	Other	120	15

* Data summarize કરીને summary() function કહેવાય.

E.x. summary(Data-frame)

* Data frame થી columns access કરીને [], [[]] અને \$ કહેવાય.

E.x. Data-frame [1]

Data-frame [["Training"]]

Data-frame \$ Training

* rows add કરીને rbind() અને columns add કરીને cbind() functions કહેવાય.

E.x. New-Data-frame1 <- rbind(Data-frame, c("Strength", 110, 110))

New-Data-frame2 <- cbind(Data-frame, steps = c(1000, 6000, 2000))

* columns અને rows બંને કરીને c() function કહેવાય.

E.x. Data-frame-New <- Data-frame [-c(1), -c(1)]

* rows અને columns જાણવા માટે dim() function કહેવાય.

E.x. dim(Data-frame)

* rows જાણવા અને columns જાણવા માટે nrow() અને ncol() functions કહેવાય.

E.x. nrow(Data-frame)

ncol(Data-frame)

* length થી જાણવા માટે length() function કહેવાય.

E.x. length(Data-frame)

* Data frames combine કરીને rbind() અને cbind() functions કહેવાય.

Factors

* Data categorize કરીને કહેવાય.

* factor() function કહેવાય અને argument થી વેક્ટર થી બનેલું કહેવાય.

E.x. music-genre <- factor(c("Jazz", "Rock", "Classic", "Rock", "Jazz"))

music-genre

* factor થી output કરીને levels() function ڪوڊ ڪري.

E.x. levels(music-genre)

* factor() function ۾ 'levels' argument ڏيکارڻ levels set ڪرڻو ضروري آهي.

E.x. music-genre <- factor(c("Jazz", "Pop", "Classic", "Rock", "Pop"),
levels = c("Classic", "Pop"))

levels(music-genre)

* factor length ڏيکارڻ length() function ڪوڊ ڪري.

E.x. length(music-genre)

* factor ڏيکارڻ access ڪرڻو indexing ڪوڊ ڪري.

E.x. music-genre[3]

* item ڏيکارڻ change ڪرڻو ۽ indexing ڪوڊ ڪري.

E.x. music-genre[3] <- "Pop"

⊛ Note : جيڪو value ڏيکارڻ لاءِ ڪو factor ڏيکارڻ level ڏيکارڻ لاءِ ڪو value ڏيکارڻ ڪوڊ ڪري error ڏيکارڻ ڪري.

E.x. music-genre[3] <- "Opera" # Error!

R Graphical Representation

Plot

- * diagram එනම් points (markers) අලුත් කරමින් plot() function යොදා.
- * parameters 2 ක් යොදා ගනියි.

parameter 1 $\Rightarrow$ X-අක්ෂයේ තත්ත්වය

parameter 2 $\Rightarrow$ Y-අක්ෂයේ තත්ත්වය

E.x. `plot(1,3)`

- * points වැඩි ගණනක් ඇති vectors යොදා ගනියි.

E.x. `plot(c(1,8), c(3,10))`

- * points sequence එනම් තිරිසන්යාම ':' operator යොදා.

E.x. `plot(1:10)`

- * plot function එනම් නම type parameter එනම් යොදා value එනම් 1 ලෙස යොදා, සියලුම points connect කර line එනම් තිරිසන්යාම කර ගනියි.

E.x. `plot(1:10, type = "l")`

- * plot එනම් label name එනම් 'main' parameter එනම්, X-අක්ෂයේ label name එනම් 'xlab' parameter එනම් Y-අක්ෂයේ label name එනම් 'ylab' parameter එනම් යොදා.

E.x. `plot(1:10, main = "My Graph", xlab = "The x-axis", ylab = "The y-axis")`

- * 'col' parameter එනම් color එනම් යොදා points වල වර්ණය තෝරා ගත හැක.

E.x. `plot(1:10, col = "red")`

- * 'cex' parameter controls points size and color is blue.
1 default value is given.

E.x. `plot(1:10, cex=2)`

- * 'pch' parameter controls point shape and color is blue. 0 to 25 different symbols are given.

E.x. `plot(1:10, pch=25, cex=2)`

Line

- * `type="l"` is line graph and color is blue.
- * line color (line color) controls and color is blue. 'col' parameter is given.

E.x. `plot(1:10, type="l", col="blue")`

- * line width and change controls 'lwd' parameter is given. Default value is 1.

E.x. `plot(1:10, type="l", lwd=2)`

- * 'lty' parameter controls line style and color is blue. 0-6 values are given and default value is 1. Solid line is given.

E.x. `plot(1:10, type="l", lwd=5, lty=3)`

available values →

- 0 → remove the line
- 1 → display a solid line
- 2 → display a dashed line
- 3 → display a dotted line
- 4 → display a dot-dashed line
- 5 → display a long-dashed line
- 6 → display a two-dashed line

* එකම graph එකේ lines තිබියදීත් පුළුල්ව plot() function එක
කරන lines() function එක යොදායි.

E.x. line1 <- c(1, 2, 3, 4, 5, 10)

line2 <- c(2, 5, 7, 8, 9, 10)

plot(line1, type="l", col="blue")

lines(line2, type="l", col="red")

Scatterplot

* plots compare තර්මය යොදාගෙන ඇත.

* මෙහි points() function යොදා ගනිති.

E.x. # day one, the age and speed of 12 cars

x1 <- c(5, 7, 8, 7, 2, 2, 9, 4, 11, 12, 9, 6)

y1 <- c(99, 86, 87, 88, 111, 103, 87, 94, 78, 77, 85, 86)

day two, the age and speed of 10 cars.

x2 <- c(2, 2, 3, 4, 5, 1, 2, 3, 4, 4)

y2 <- c(99, 86, 86, 86, 111, 100, 90, 78, 76, 76)

plot(x1, y1, main="Observation of cars",
xlab="Car age", ylab="Car speed", col="red",
cex=2)

points(x2, y2, col="blue", cex=2)

Pie charts

- ★ pie charts නිර්මාණය කිරීමට `pie()` function යොදා ගනී.

E.x. `x <- c(10, 20, 30, 40)`

`pie(x)`

- ★ pie chart එකෙහි starting angle එක `'init.angle'` parameter එක මගින් තෝරා ගත හැක. Angle එක ඉහත සඳහන් වලින් කළයුතුය; default value එක 0 වේ.

E.x. `x <- c(10, 20, 30, 40)`

`pie(x, init.angle = 90)`

- ★ pie chart එකට labels යෙදවීමට `'label'` parameter එක යොදා ගනී.

E.x. `x <- c(10, 20, 30, 40)`

`mylabel <- c("Apples", "Bananas", "Cherries", "Mangoes")`

`pie(x, label = mylabel, main = "Fruits")`

- ★ `'col'` parameter එකෙන් pies වලට වර්ණ යොදා ගත හැක.

E.x. `colors <- c("blue", "yellow", "red", "green")`

`pie(x, label = mylabel, main = "Fruits", col = colors)`

- ★ `legend()` function එක මගින් pies සඳහා explanation එකක් යොදා ගත හැක.

E.x. `legend("bottomright", mylabel, fill = colors)`

Bar charts

- ★ verticle bar chart එකක් නිර්මාණය කිරීමට `barplot()` function යොදා ගනී.

E.x. `x <- c("A", "B", "C", "D")`

`y <- c(2, 4, 6, 8)`

`barplot(y, names.arg = x)`

④ Note : X variable થી X-ફેક્ટરના values મેળવે છે.

Y variable થી Y-ફેક્ટરના values મેળવે છે.

names.arg parameter થી X-ફેક્ટરના observations
વેળા મેળવે names નામ મેળવે.

★ 'col' parameter થી રંગના bar color થી નામ મેળવે.

E.x. `barplot(y, names.arg = X, col = "red")`

★ bar texture થી ઘેરે સંબંધે 'density' parameter થી મેળવે.

E.x. `barplot(y, names.arg = X, density = 10)`

★ 'width' parameter થી રંગના bar width થી ઘેરે મેળવે.

E.x. `barplot(y, names.arg = X, width = c(1, 2, 3, 4))`

★ horizontal bars સંબંધે "horiz = TRUE" મેળવે.

E.x. `barplot(y, names.arg = X, horiz = TRUE)`